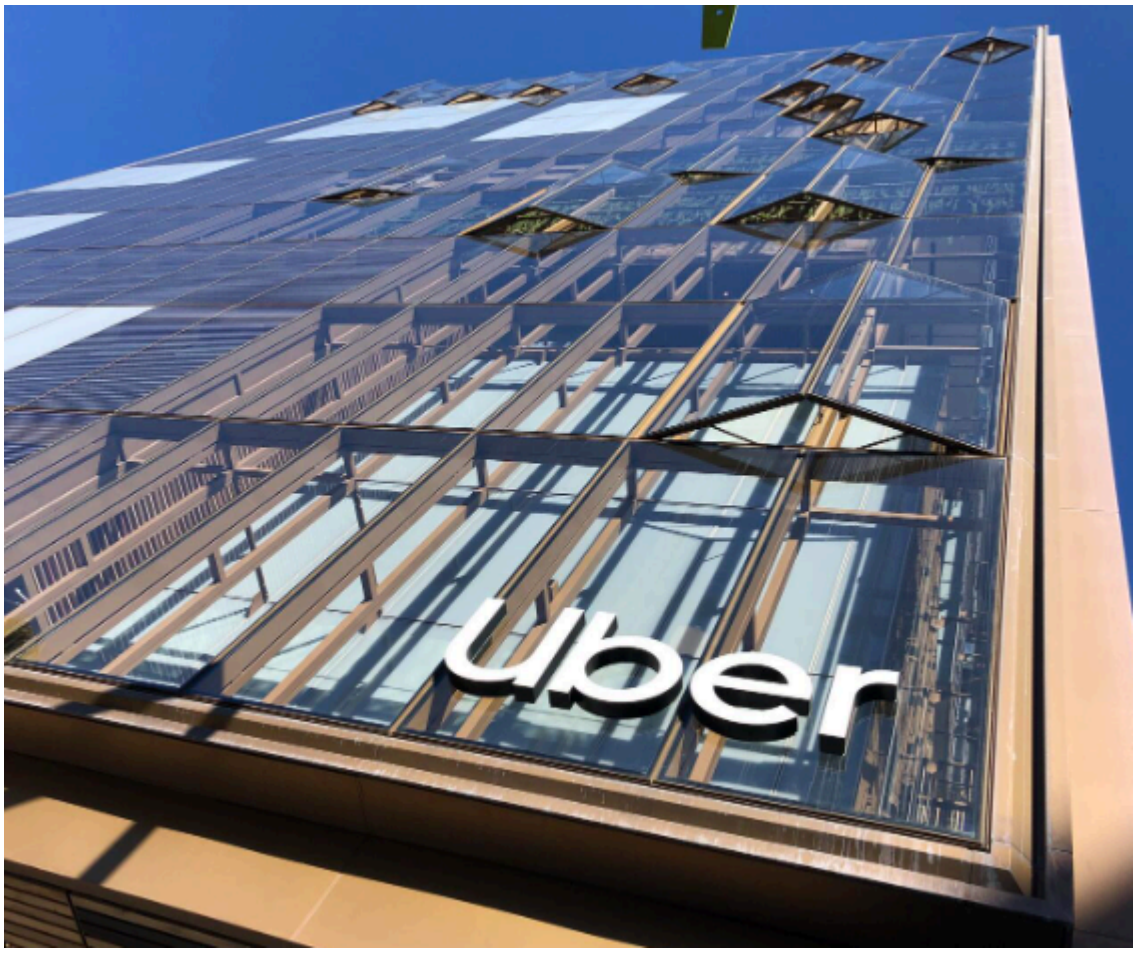
“ राम - राम ”



Find the number of subarrays such that their maximum is K -

Ex : [8 2 1 3 4 5 1 10] , K = 3

Output = 3. [3],[1 3],[2 1 3]

```
Brute Force :
Generate all possible subarrays and check if its max
is k then increment the count
T.C = O(N^2)
Int count = 0;
for(int i=0; i<n; i++){
     max = a[i];
     for(int j=i; j<n; j++){
          max = Math.max(max, a[j]);
          if(max == k)count++;
     }
};
```

Optimized :
If my current number is k then number of subarrays containing k is x*y + (x+y) + 1
where x is count of numbers before k and y is count of numbers after k
[8 2 1 3 0 1 2 4 5 1 10]
In this example here count of numbers before k is 2 and after k is 3 which are making a subarray with k

So total subarrays for x = 2 and y = 3 containing k => 2*3 + (2+3) + 1 = 12
(2,1,3),(2,1,3,1),(2,1,3,0,1),(2,1,3,0,1,2),(1,3),
(1,3,0),(1,3,0,1),(1,3,0,1,2),(3,0),(3,0,1),
(3,0,1,2),(3)

When we have an array like this [8 2 1 3 0 1 10 3 1 2]

Then also our formula will work b'coz count of x and y on left and right resp. of k does not include k so here there will be no repetition of count
So k at index 4 has contribution of 2*2 + 2 + 2 + 1 to total count And now contribution of k at index 8 has contribution of 0*2 + 0 + 2 + 1
So total number of subarrays having k as their max = 9+3 = 12

When we have an array like this [8 2 1 3 0 1 3 1]
Now if we apply that formula here then the count of subarrays will be
For k at
⇒ idx 4 = 2*4 + 2 + 4 + 1 = 15
⇒ idx 7 = 5*1 + 5 + 1 + 1 = 12
Total = 27
So to fix this what we can do is we will count all the numbers <= k from left and from right we will

count only those numbers < k that can make subarray with k
Hence it will fix the problem of repeated count
So corrected ans
⇒ at idx 4 = 2*2 + 2 + 2 + 1 = 9
⇒ at idx 7 = 5*1 + 5 + 1 + 1 = 12
Total = 21

Find the number of subarrays having min as k1 and max as k2

First try to find the good subarrays
A subarray is good when we can say that this subarray has min as k1 and max as k2
Ex : [] , k1 = 2, k2 = 5

